

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 3

Dasar Compose Multiplatform

Menyusun layar daftar laporan dengan composable, layout, modifier, dan LazyColumn

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 3. Tahun 2026

Komponen	Keterangan
Pertemuan	3
Capaian	CPMK0501, menerapkan konsep pemrograman dalam bentuk UI yang terstruktur dengan layout yang tepat
Waktu belajar	sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 3

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: data sudah ada, layarnya belum
3	Langkah 1: Cara berpikir deklaratif
4	Langkah 2: Aturan main composable function
5	Langkah 3: Tiga wadah dasar dan Column
6	Langkah 4: Row dan weight
7	Langkah 5: Box untuk menumpuk
8	Langkah 6: Modifier dirangkai, dan urutannya berarti
9	Langkah 7: Text, tombol, gambar, dan ikon
10	Langkah 8: Komponen yang bisa dipakai ulang
11	Langkah 9: Mengolah daftar sebelum ditampilkan
12	Langkah 10: Daftar panjang dengan LazyColumn
13	Latihan mandiri
14	Bawa ke proyekmu
15	Rangkuman
16	Cek pemahaman
17	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 3. Di Modul 2 kita sudah punya data class Laporan, sealed class StatusLaporan, dan fungsi `Label()`. Semuanya masih hidup di konsol. Di modul ini, data itu kita tampilkan sebagai layar daftar yang rapi, dengan satu kode untuk Android dan desktop.

Modul ini terdiri dari tiga bagian. Langkah 1 dan 2 membahas cara berpikir Compose. Langkah 3 sampai 6 membahas cara menyusun layar dengan Column, Row, Box, dan Modifier. Langkah 7

sampai 10 membahas komponen, cara membuatnya bisa dipakai ulang, dan cara menampilkan daftar panjang.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menjelaskan beda cara deklaratif dan imperatif, serta aturan composable function	Langkah 1 dan 2
2	Menyusun layar dengan Column, Row, Box, dan Modifier dengan urutan yang tepat	Langkah 3 sampai 6
3	Menulis komponen yang bisa dipakai ulang, mengolah daftar, lalu menampilkannya dengan LazyColumn	Langkah 7 sampai 10

Capaian ini mendukung **CPMK0501**, dalam bentuk UI yang terstruktur dengan layout yang tepat. Yang diamati minggu ini adalah kerapian tampilan dan ketepatan urutan modifier.

Yang perlu disiapkan

Proyek Laporkampus yang sudah berisi model data dari Modul 2. Semua kode di modul ini ditulis di **commonMain/ui**. Jalankan dengan target desktop supaya hasilnya cepat terlihat setiap kali kalian mengubah kode.

PENTING Import yang paling sering dibutuhkan

Android Studio biasanya menawarkan import otomatis saat kalian menekan Alt+Enter pada nama yang merah. Kalau ragu, ingat empat paket ini:

`androidx.compose.foundation.layout.*` untuk Column, Row, Box, dan padding;

`androidx.compose.material3.*` untuk Text, Button, dan Card; `androidx.compose.ui.unit.dp` dan `sp` untuk ukuran; serta `androidx.compose.ui.graphics.Color` untuk warna.

PEMBUKA

Masalah pembuka: data sudah ada, layarnya belum

Bayangkan layar Laporkampus yang ingin kita buat hari ini. Di bagian atas ada judul aplikasi dan jumlah laporan aktif. Di bawahnya ada daftar kartu laporan. Setiap kartu berisi judul dan lokasi di kiri, lalu label status di kanan. Di pojok kanan bawah ada tombol Lapork.

Bagian layar	Contoh isi
Judul	Laporkampus
Keterangan	3 laporan aktif
Kartu 1	Proyektor mati, GUK 1 R201, label Baru
Kartu 2	AC mati, GUK 2 R305, label Diproses
Kartu 3	Keran bocor, Perpustakaan, label Selesai
Tombol	Lapork, di pojok kanan bawah

TEBAK DULU Coba pikirkan

Lihat susunan layar di atas. Bagian mana yang tersusun dari atas ke bawah, mana yang dari kiri ke kanan, dan mana yang bertumpuk?

JAWABAN

Layar keseluruhan tersusun dari atas ke bawah: judul, keterangan, lalu kartu-kartu. Isi satu kartu tersusun dari kiri ke kanan: teks di kiri, label di kanan. Label status yang ditaruh di atas foto adalah contoh susunan bertumpuk. Tiga pola ini punya nama di Compose: **Column**, **Row**, dan **Box**. Ketiganya menjadi kerangka modul ini.

LANGKAH 1

Cara berpikir deklaratif

Sebelum Compose, tampilan Android dibuat dengan XML, lalu diubah dari kode Kotlin atau Java langkah demi langkah. Cara ini disebut **imperatif**: kita memerintah tampilan satu per satu.

Imperatif, Android View lama

```

val judul = findViewById<TextView>(R.id.judul)
judul.text = laporan.judul
judul.visibility = View.VISIBLE
// status berubah? ubah lagi satu per satu
judul.text = laporanBaru.judul

```

Masalahnya, setiap perubahan data harus kita terjemahkan sendiri menjadi perintah ke tampilan, dan mudah ada yang terlewat. Compose memakai cara **deklaratif**: kita cukup mendeskripsikan seperti apa layar seharusnya untuk data tertentu.

Deklaratif, Compose

```

@Composable
fun Judul(laporan: Laporan) {
    Text(text = laporan.judul)
}
// data berubah? Compose memanggil ulang Judul

```

Di Compose, UI adalah fungsi dari data. Kalau datanya berubah, fungsinya dipanggil ulang, dan tampilannya ikut berubah. Proses memanggil ulang ini disebut **recomposition**, dan kita bedah lengkap di Modul 4.

ANALOGI Memesan di warung

Di warung, kita cukup bilang pecel lele satu, sambal dipisah. Kita tidak mendikte cara menyalakan kompor, kapan membalik lele, atau berapa lama menggoreng. Compose bekerja seperti itu: kita menyebut hasil yang kita mau, Compose yang mengurus cara menggambarinya.

LANGKAH 2

Aturan main composable function

Fungsi yang menggambar tampilan disebut **composable function**. Bentuknya fungsi Kotlin biasa dengan anotasi `@Composable`.

```
commonMain/ui/Sapaan.kt
```

```
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.unit.sp

@Composable
fun SapaanPelapor(nama: String) {
    Text(
        text = "Halo, $nama",
        fontSize = 24.sp
    )
}

@Composable
fun App() {
    SapaanPelapor(nama = "Rina")
}
```

Aturan	Alasannya
Diberi anotasi <code>@Composable</code>	Compose perlu tahu fungsi mana yang menggambar UI
Dipanggil dari composable lain	Hanya di sana Compose bisa melacak perubahannya
Tidak mengembalikan apa pun	Fungsi ini memancarkan UI ke layar, bukan membuat objek tampilan
Nama memakai PascalCase	Supaya terbaca sebagai komponen, seperti Text dan Button

dp dan sp

Ukuran di Compose memakai dua satuan. **dp** (density-independent pixel) dipakai untuk jarak dan ukuran benda, supaya tampil sama besar di HP dengan kerapatan layar berbeda. **sp** (scale-independent pixel) dipakai khusus untuk ukuran huruf, karena ikut membesar kalau pengguna memperbesar ukuran huruf di pengaturan HP.

ANALOGI Cetakan kue

Composable function seperti cetakan kue. Cetakannya ditulis sekali, lalu bisa dipakai berkali-kali dengan adonan yang berbeda. `SapaanPelapor` adalah cetakannya, dan nama Rina adalah adonannya.

TEBAK DULU

Apa yang terjadi kalau `SapaanPelapor` dipanggil dari `fun main()` biasa yang tidak diberi anotasi `@Composable`?

JAWABAN

Kode gagal dikompilasi dengan pesan **@Composable invocations can only happen from the context of a @Composable function**. Pesan ini akan sering kalian lihat minggu ini. Obatnya selalu sama: pastikan pemanggilnya juga composable.

LANGKAH 3

Tiga wadah dasar dan Column

Hampir semua layar bisa dipecah menjadi kombinasi tiga wadah:

Wadah	Menyusun anak	Dipakai di Laporkampus untuk
Column	Dari atas ke bawah	Layar daftar
Row	Dari kiri ke kanan	Isi satu kartu
Box	Bertumpuk di tempat yang sama	Label status di atas foto

ANALOGI Rak, antrean, dan tumpukan stiker

Column seperti rak buku bertingkat: isinya disusun dari atas ke bawah. Row seperti orang yang mengantre berjejer dari kiri ke kanan. Box seperti stiker yang ditempel di laptop: yang ditempel belakangan menutupi yang lebih dulu.

Column untuk layar daftar

commonMain/ui/LayarDaftar.kt

```
@Composable
fun LayarDaftar(semua: List<Laporan>) {
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Text("Laporkampus", fontSize = 28.sp)
        Text("${semua.size} laporan aktif", color = Color.Gray)
        semua.take(3).forEach { Text(it.judul) }
    }
}
```

Untuk sementara setiap laporan ditampilkan sebagai satu Text berisi judulnya. Di Langkah 8, baris ini kita ganti dengan kartu yang lengkap.

Parameter	Pilihan	Artinya
<code>verticalArrangement</code>	Top, Center, Bottom, SpaceBetween, spacedBy	Cara membagi ruang vertikal di antara anak
<code>horizontalAlignment</code>	Start, CenterHorizontally, End	Posisi anak dari kiri ke kanan

`spacedBy(12.dp)` memberi jarak tetap 12 dp di antara setiap anak. `SpaceBetween` membagi sisa ruang secara merata, sehingga anak pertama menempel di atas dan anak terakhir menempel di bawah. Coba ganti nilainya dan jalankan, supaya kalian melihat sendiri bedanya.

Untuk mencoba, buat data contoh dan panggil dari App:

commonMain/App.kt

```
val contoh = listOf(
    Laporan(1, "Proyektor mati", "GKU 1 R201", null),
    Laporan(2, "AC mati", "GKU 2 R305", null, StatusLaporan.Diproses("Pak Joko")),
    Laporan(3, "Keran bocor", "Perpustakaan", null, StatusLaporan.Selesai("Keran diganti"))
)

@Composable
fun App() = MaterialTheme {
    LayarDaftar(contoh)
}
```

LANGKAH 4

Row dan weight

Row menyusun anak dari kiri ke kanan. Kita pakai untuk baris lokasi yang berisi ikon, nama lokasi, dan waktu lapor.

commonMain/ui/BarisLokasi.kt

```
@Composable
fun BarisLokasi(lokalasi: String, waktu: String) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(Icons.Default.Place, contentDescription = null)
        Text(lokalasi, modifier = Modifier.weight(1f))
        Text(waktu, color = Color.Gray)
    }
}
```

`Modifier.weight(1f)` membuat teks lokasi mengambil semua sisa ruang, sehingga waktu terdorong ke ujung kanan. Tanpa weight, ketiganya menempel di kiri dan sisa ruang kosong di kanan.

ANALOGI Penumpang yang mengambil sisa bangku

Bayangkan bangku panjang di angkot. Dua penumpang duduk dengan lebar badannya masing-masing, sedangkan satu penumpang diberi `weight`: ia mengisi semua sisa bangku. Kalau dua anak sama-sama diberi `weight(1f)`, sisa ruang dibagi rata. Kalau satu diberi `weight(2f)` dan yang lain `weight(1f)`, pembagiannya dua banding satu.

PENTING weight hanya ada di Row dan Column

`weight` hanya bisa dipakai pada anak langsung dari `Row` atau `Column`, karena hanya wadah itu yang tahu cara membagi sisa ruang. Kalau kalian menuliskannya di luar, Android Studio menandainya merah.

LANGKAH 5

Box untuk menumpuk

Box menumpuk anak di tempat yang sama. Kita pakai untuk menaruh label status di pojok kanan atas foto laporan.

```
commonMain/ui/FotoLaporan.kt
```

```
@Composable
fun FotoLaporan(laporan: Laporan) {
    Box(
        modifier = Modifier.fillMaxWidth().height(160.dp),
        contentAlignment = Alignment.TopEnd
    ) {
        FotoAtauKosong(laporan.fotoUrl)
        LabelStatus(laporan.status)
    }
}
```

Anak yang ditulis belakangan digambar paling atas. Foto ditulis dulu, baru label status, sehingga label tampil di atas foto. `contentAlignment` menentukan posisi anak di dalam Box. Ada sembilan pilihan, dari `TopStart` sampai `BottomEnd`.

`FotoAtauKosong` dan `LabelStatus` belum ada, jadi kita buat sekarang. Foto dari internet baru kita tampilkan di Pertemuan 6, jadi untuk sementara kotaknya hanya berisi teks.

```
commonMain/ui/LabelStatus.kt
```

```

@Composable
fun FotoAtauKosong(url: String?) {
    Box(
        modifier = Modifier.fillMaxSize().background(Color(0xFFF1EDE1)),
        contentAlignment = Alignment.Center
    ) {
        Text(if (url == null) "Belum ada foto" else "Foto tersedia", color =
Color.Gray)
    }
}

@Composable
fun LabelStatus(status: StatusLaporan) {
    val (teks, warna) = when (status) {
        StatusLaporan.Baru → "Baru" to Color(0xFFE6E2D6)
        is StatusLaporan.Diproses → "Diproses" to Color(0xFFF6EDCF)
        is StatusLaporan.Selesai → "Selesai" to Color(0xFFDCEFD6)
    }
    Text(
        text = teks,
        fontSize = 12.sp,
        modifier = Modifier
            .background(warna, RoundedCornerShape(50))
            .padding(horizontal = 12.dp, vertical = 4.dp)
    )
}

```

Ada dua hal baru di `LabelStatus`. Pertama, `"Baru" to warna` membuat pasangan dua nilai, dan `val (teks, warna) = ...` membongkar pasangan itu menjadi dua variabel sekaligus. Kedua, `RoundedCornerShape(50)` membuat sudut membulat 50 persen, sehingga labelnya berbentuk pil.

TEBAK DULU

Apa yang terjadi kalau urutan `FotoAtauKosong` dan `LabelStatus` di dalam `Box` ditukar?

JAWABAN

Labelnya tertutup foto. `FotoAtauKosong` mengisi seluruh `Box` dengan `fillMaxSize()`, dan karena ditulis belakangan, ia digambar paling atas dan menutupi label.

LANGKAH 6

Modifier dirangkai, dan urutannya berarti

Modifier mengatur ukuran, jarak, warna latar, bingkai, dan interaksi sebuah komponen. Beberapa modifier bisa dirangkai dengan titik.

```
commonMain
```

```
Text(
    text = "Diproses",
    modifier = Modifier
        .padding(16.dp)
        .background(Color(0xFFF6EDCF))
        .padding(8.dp)
        .clickable { bukaDetail() }
)
```

Baris	Artinya
<code>padding(16.dp)</code>	Jarak di luar warna
<code>background(...)</code>	Warna latar
<code>padding(8.dp)</code>	Jarak di dalam warna
<code>clickable { }</code>	Area yang bisa diketuk

Modifier dibaca dari atas ke bawah dan diterapkan berurutan. Setiap modifier membungkus hasil modifier sebelumnya.

ANALOGI **Membungkus kado**

Bayangkan membungkus kado lapis demi lapis. Kalau kalian memberi ruang kosong dulu lalu membungkus dengan kertas kado, ruang kosong itu ikut terbungkus dan kertasnya jadi lebar. Kalau kalian membungkus dulu lalu memberi ruang, ruang itu ada di luar kertas. Urutan yang berbeda menghasilkan bentuk yang berbeda, walaupun bahannya sama.

TEBAK DULU

Kedua kode ini memakai modifier yang sama dengan urutan berbeda. Gambarkan di kertas, di mana warna kuning muncul pada A, dan di mana pada B?

A

```
Text("Diproses", Modifier.padding(16.dp).background(Kuning))
```

B

```
Text("Diproses", Modifier.background(Kuning).padding(16.dp))
```

JAWABAN

Pada **A**, jarak 16 dp dipasang dulu dan tetap transparan, lalu warna kuning hanya menempel di sekitar teks. Pada **B**, warna kuning dipasang dulu, lalu jarak ditambahkan di dalamnya, sehingga kuning ikut melebar 16 dp ke segala arah. Pegangan praktisnya: padding sebelum background berperan sebagai jarak di luar, padding sesudah background berperan sebagai jarak di dalam.

Modifier yang paling sering dipakai

Kelompok	Contoh	Di Laporan Kampus
Ukuran	<code>fillMaxWidth(), size(48.dp), height(160.dp)</code>	Foto laporan selebar layar
Jarak	<code>padding(16.dp), padding(horizontal = 8.dp)</code>	Ruang di dalam kartu
Latar	<code>background(warna, RoundedCornerShape(50))</code>	Label status berbentuk pil
Bingkai	<code>border(1.dp, Color.Gray), clip(CircleShape)</code>	Foto profil pelapor bulat
Interaksi	<code>clickable { }</code>	Kartu bisa diketuk untuk membuka detail

Simpan tabel ini sebagai rujukan saat mengerjakan latihan.

LANGKAH 7

Text, tombol, gambar, dan ikon

Text

```
commonMain
```

```
Text(
    text = laporan.judul,
    fontWeight = FontWeight.Bold,
    maxLines = 1,
    overflow = TextOverflow.Ellipsis
)
```

`maxLines = 1` dan `overflow = TextOverflow.Ellipsis` menjaga judul laporan yang panjang tetap satu baris dan diakhiri titik tiga, sehingga tidak merusak tata letak kartu.

Tombol

Komponen	Dipakai untuk	Contoh
Button	Aksi utama	Kirim laporan
OutlinedButton	Aksi kedua	Simpan draf
TextButton	Aksi ringan	Batal
FloatingActionButton	Satu aksi terpenting di layar	Buat laporan baru

commonMain

```
FloatingActionButton(onClick = { bukaForm() }) {
    Icon(Icons.Default.Add, contentDescription = "Buat laporan")
}
```

Satu layar sebaiknya punya satu aksi utama. Di Laporkampus, aksi utamanya adalah tombol Lapork.

Gambar dan ikon

commonMain

```
Image(
    painter = painterResource(Res.drawable.logo_kampus),
    contentDescription = "Logo kampus",
    modifier = Modifier.size(48.dp).clip(CircleShape)
)
Icon(
    imageVector = Icons.Default.Warning,
    contentDescription = "Prioritas tinggi",
    tint = Color(0xFF8C1515)
)
```

Gambar lokal ditaruh di folder **composeResources/drawable**. Setelah build, Compose membuat kelas `Res` sehingga gambar bisa dipanggil sebagai `Res.drawable.logo_kampus`. Foto laporan dari internet butuh library tambahan seperti Coil, dan kita pasang saat data mulai datang dari server di Pertemuan 6.

PENTING contentDescription untuk semua pengguna

`contentDescription` dibacakan oleh pembaca layar untuk pengguna tunanetra. Isi dengan arti gambarnya, misalnya Prioritas tinggi, bukan Ikon segitiga merah. Kalau gambar hanya hiasan, isi dengan null supaya tidak dibacakan.

LANGKAH 8

Komponen yang bisa dipakai ulang

KartuLaporan, menyatukan semuanya

commonMain/ui/KartuLaporan.kt

```
@Composable
fun KartuLaporan(laporan: Laporan) {
    Card(elevation = CardDefaults.cardElevation(2.dp)) {
        Row(
            modifier = Modifier.fillMaxWidth().padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(Modifier.weight(1f)) {
                Text(laporan.judul, fontWeight = FontWeight.Bold)
                Text(laporan.lokasi, color = Color.Gray)
            }
            LabelStatus(laporan.status)
        }
    }
}
```

Card berisi Row, dan Row berisi Column serta label. Persis seperti hasil diskusi di masalah pembuka. Sekarang buka LayarDaftar dan ganti `Text(it.judul)` dengan `KartuLaporan(it)`, lalu jalankan.

Fungsi juga bisa dikirim sebagai nilai

Di Modul 2 kita sudah memakai lambda di `filter { }`. Sekarang kita lihat dari sisi lain: fungsi bisa disimpan di variabel dan dikirim sebagai parameter.

Main.kt

```
fun ulang(kali: Int, aksi: (Int) → Unit) {
    for (i in 1..kali) aksi(i)
}

fun main() {
    ulang(3) { println("Laporan ke-$it") }
}
```

Keluaran

```
Laporan ke-1
Laporan ke-2
Laporan ke-3
```

Penulisan

Artinya

```
() → Unit
```

Fungsi tanpa masukan dan tanpa hasil

Penulisan	Artinya
<code>(Int) → Unit</code>	Fungsi dengan satu masukan Int, tanpa hasil
<code>{ ... }</code>	Lambda, fungsi tanpa nama
<code>it</code>	Nama otomatis untuk satu masukan

Kalau lambda adalah argumen terakhir, ia boleh ditulis di luar kurung, seperti `ulang(3) { }`. Inilah sebabnya kita bisa menulis `Column { }` dan `Row { }`: blok di dalam kurung kurawal itu adalah lambda.

Parameter membuat komponen bisa dipakai ulang

```
commonMain/ui/KartuLaporan.kt

@Composable
fun KartuLaporan(
    laporan: Laporan,           // wajib diisi
    ringkas: Boolean = false,    // punya nilai bawaan
    onKlik: () → Unit = {}      // dipanggil saat diketuk
) {
    Card(
        modifier = Modifier.fillMaxWidth().clickable { onKlik() },
        elevation = CardDefaults.cardElevation(2.dp)
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(Modifier.weight(1f)) {
                Text(laporan.judul, fontWeight = FontWeight.Bold)
                if (!ringkas) Text(laporan.lokasi, color = Color.Gray)
            }
            LabelStatus(laporan.status)
        }
    }
}
```

Sekarang satu komponen bisa dipakai dengan banyak cara:

```
commonMain

KartuLaporan(laporan)
KartuLaporan(laporan, ringkas = true)
KartuLaporan(laporan, onKlik = { bukaDetail(laporan.id) })
```

Perhatikan bahwa `KartuLaporan` tidak tahu apa yang terjadi saat diketuk. Ia hanya memanggil `onKlik`, dan pemanggilnya yang memutuskan.

ANALOGI Bel pintu

Komponen dengan onKlik seperti bel pintu. Bel hanya berbunyi saat ditekan. Yang memutuskan apa yang dilakukan, membuka pintu atau pura-pura tidak di rumah, adalah pemilik rumah. Pola ini sangat penting di Modul 4 dan 5: komponen hanya melaporkan kejadian, keputusan ada di luar.

LANGKAH 9

Mengolah daftar sebelum ditampilkan

Sebelum ditampilkan, daftar laporan sering perlu disaring, diurutkan, atau dikelompokkan. Kotlin punya banyak fungsi koleksi untuk itu.

```
commonMain/ui/LayarDaftar.kt
```

```
val baru = semua.filter { it.status is StatusLaporan.Baru }
val judul = semua.map { it.judul }
val terbaru = semua.sortedByDescending { it.id }
val perLokasi = semua.groupBy { it.lokasi }
val adaFoto = semua.count { it.fotoUrl != null }
```

Fungsi	Hasilnya	Contoh hasil
<code>filter</code>	Daftar yang lolos syarat	Laporan berstatus Baru saja
<code>map</code>	Daftar baru hasil ubahan	Daftar judul saja
<code>sortedByDescending</code>	Daftar terurut menurun	Laporan terbaru di atas
<code>groupBy</code>	Peta kelompok per kunci	Laporan dikelompokkan per gedung
<code>count</code>	Jumlah yang lolos syarat	Banyaknya laporan yang berfoto
<code>take</code>	Beberapa item pertama	20 laporan teratas

PENTING Daftar asli tidak berubah

Semua fungsi di atas membuat daftar baru. Daftar `semua` tetap utuh. Ini sejalan dengan kebiasaan memakai `val` dan `copy` di Modul 2.

CONTOH Contoh sehari-hari

Bayangkan daftar hadir kelas. filter seperti memilih mahasiswa yang hadir saja. map seperti menyalin namanya saja tanpa NIM. sortedBy seperti mengurutkan berdasarkan abjad. groupBy seperti memisahkan per angkatan. count seperti menghitung yang hadir. Daftar hadir aslinya tetap tersimpan di meja dosen.

TEBAK DULU

perLokasi di atas bertipe apa?

JAWABAN

Map<String, List<Laporan>>. Kuncinya nama lokasi, isinya daftar laporan di lokasi itu. Bentuk ini berguna untuk menampilkan laporan yang dikelompokkan per gedung.

LANGKAH 10

Daftar panjang dengan LazyColumn

Sejauh ini kita hanya menampilkan tiga laporan dengan Column dan forEach. Bagaimana kalau laporannya ratusan?

commonMain/ui/DaftarLaporan.kt

```
@Composable
fun DaftarLaporan(
    semua: List<Laporan>,
    onPilih: (Laporan) → Unit
) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(sema, key = { it.id }) { laporan →
            KartuLaporan(laporan, onKlik = { onPilih(laporan) })
        }
    }
}
```

LazyColumn hanya menggambar kartu yang sedang terlihat di layar, lalu memakai ulang tempatnya saat pengguna menggulir. Parameter **key** membantu Compose mengenali laporan yang sama walaupun urutannya berubah, misalnya setelah daftar diurutkan ulang.

ANALOGI Petugas perpustakaan

Column dengan `forEach` seperti meminta petugas perpustakaan menumpuk seluruh koleksi buku di meja kalian sekaligus. Mejanya penuh dan lama menunggu. `LazyColumn` seperti petugas yang hanya mengambilkan buku yang sedang kalian baca, lalu menukarnya saat kalian pindah halaman. `key` seperti nomor panggil buku: walaupun rak disusun ulang, buku yang sama tetap dikenali.

Coba dengan 200 laporan palsu:

commonMain/App.kt

```
val banyak = List(200) { i →
    Laporan(i, "Laporan ke- $i$ ", "GKU  $i \% 3 + 1$ ", null)
}

@Composable
fun App() = MaterialTheme {
    DaftarLaporan(banyak, onPilih = { println("Dipilih # $\{it.id\}$ ") })
}
```

Gulir dengan cepat. Daftarnya tetap mulus walaupun berisi 200 kartu.

TEBAK DULU

LaporKampus punya 500 laporan. Kalau semuanya ditampilkan dengan `Column` dan `forEach`, apa yang terjadi saat layar pertama kali dibuka? Kapan `Column` dengan `forEach` masih pantas dipakai?

JAWABAN

`Column` menggambar kelima ratus kartu sekaligus, termasuk yang tidak terlihat, sehingga layar lambat terbuka dan memori boros. `Column` juga tidak bisa digulir kecuali diberi `Modifier.verticalScroll(rememberScrollState())`. `Column` dengan `forEach` masih pantas untuk daftar pendek yang jumlahnya pasti, misalnya tiga tombol kategori atau lima baris info di halaman profil.

Menyatukan menjadi satu layar

Langkah terakhir: gabungkan judul, daftar, dan tombol Lapor dengan **Scaffold**, yaitu kerangka layar Material yang sudah menyediakan tempat untuk tombol mengambang.

commonMain/ui/LayarUtama.kt

```

@Composable
fun LayarUtama(semua: List<Laporan>) {
    val terbaru = semua.sortedByDescending { it.id }
    Scaffold(
        floatingActionButton = {
            FloatingActionButton(onClick = { /* Modul 5 */ }) {
                Icon(Icons.Default.Add, contentDescription = "Buat Laporan")
            }
        }
    ) { padding →
        Column(Modifier.padding(padding)) {
            Text("LaporKampus", fontSize = 28.sp, modifier = Modifier.padding(16.dp))
            DaftarLaporan(terbaru, onPilih = { })
        }
    }
}

```

`padding` di dalam Scaffold adalah ruang yang dipakai oleh bagian Scaffold sendiri. Selalu teruskan ke isi layar supaya tidak ada yang tertutup.

Komponen lain yang akan sering kita temui

Komponen	Gunanya	Contoh di LaporKampus
<code>TextField</code>	Masukan teks	Judul laporan, dibahas lengkap di Modul 4
<code>Checkbox</code>	Pilihan ya atau tidak	Tandai laporan sudah dibaca
<code>Switch</code>	Hidup atau mati	Notifikasi status laporan
<code>RadioButton</code>	Satu dari beberapa pilihan	Kategori kerusakan
<code>Slider</code>	Nilai dalam rentang	Tingkat keparahan 1 sampai 5
<code>CircularProgressIndicator</code>	Sedang memuat	Menunggu daftar dari server

Sebagian besar komponen ini butuh state supaya berfungsi, dan itu topik Modul 4.

PRAKTIK

Latihan mandiri

Ada tiga latihan di repo mata kuliah. Latihan 1 dan 3 Compose adalah latihan yang diobservasi.

Latihan	Lokasi	Yang dilatih
Compose 1	P3 Compose Multiplatform Basics Hands-on, Handson1ProfileCard.kt	Avatar bulat, nama tebal, dan bio abu-abu di dalam Card
Compose 3	P3 Compose Multiplatform Basics Hands-on, Handson3ProductList.kt	Kartu berisi kotak warna, nama, dan harga, diulang untuk setiap produk
Kotlin Dasar	Kotlin Dasar, P4 Collections and co. Hands-on, handson1-latihan	Nama produk di atas harga tertentu, terurut dari yang termurah, dengan filter, sortedBy, dan map

Latihan 2 Compose, yaitu form login, boleh dikerjakan kalau waktu tersisa.

CONTOH Petunjuk latihan Compose 1

Pecah dulu kartunya di kertas: Card berisi Row, Row berisi avatar dan Column, Column berisi nama dan bio. Avatar bulat dibuat dengan

`Modifier.size(64.dp).clip(CircleShape).background(warna)`. Perhatikan urutannya: size dulu, lalu clip, lalu background. Coba tukar clip dan background, lalu lihat apa yang terjadi.

CONTOH Petunjuk latihan Compose 3

Polanya mirip KartuLaporan di Langkah 8. Kotak warna di kiri, Column berisi nama dan harga di tengah dengan `weight(1f)`. Kalau sudah selesai, tantang diri kalian: ganti `forEach` dengan `LazyColumn` dan jelaskan bedanya.

CONTOH Petunjuk latihan Kotlin Dasar

Rangkai fungsi koleksi seperti Langkah 9: `filter` untuk harga di atas batas, `sortedBy` untuk urutan termurah, lalu `map` untuk mengambil namanya saja. Urutan filter sebelum `sortedBy` membuat program mengurutkan lebih sedikit data.

Yang diamati

Level	Skor	Yang terlihat
Sangat baik	85 sampai 100	Kartu profil dan daftar produk rapi, urutan modifier tepat, daftar memakai LazyColumn
Baik	70 sampai 84	Keduanya tampil, ada jarak atau perataan yang kurang tepat
Cukup	55 sampai 69	Hanya satu yang tampil, layout berantakan
Kurang	di bawah 55	Belum ada tampilan yang jalan

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk CPMK0501. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai. Urutan modifier adalah hal yang paling sering membedakan level Sangat baik dan Baik.

Kalau error

Pesan error	Penyebab	Yang dicoba
@Composable invocations can only happen from the context of a @Composable function	Memanggil composable dari fungsi biasa	Beri anotasi @Composable pada pemanggilnya
Unresolved reference: dp atau sp	Import belum ada	<code>import androidx.compose.ui.unit.dp</code> dan <code>sp</code>
Type mismatch: expected Color	Color dari paket yang salah	Pakai <code>androidx.compose.ui.graphics.Color</code>
Unresolved reference: Icons	Import ikon belum lengkap	Tambahkan import Icons dan ikon yang dipakai, atau dependency ikon Material di <code>build.gradle.kts</code>
Unresolved reference: Res	Kelas resource belum dibuat	Build proyek sekali setelah menambah gambar di <code>composeResources</code>

PENTING Jebakan Color di target desktop

Di target desktop, Android Studio kadang otomatis mengimpor `java.awt.Color`. Hasilnya error Type mismatch yang membingungkan. Periksa bagian import di atas berkas setiap kali error ini muncul.

PRAKTIK

Bawa ke proyekmu

Ini layar pertama dari lima layar wajib di Bank Proyek. Kerjakan di repository kelompok, folder commonMain/ui.

1. **Kartu entitas.** Composable kartu untuk entitas utama tema kalian, dengan label status dari sealed class minggu lalu.
2. **Daftar data contoh.** LazyColumn berisi minimal sepuluh data contoh dari data class minggu lalu.
3. **Satu aksi utama.** Tombol atau FAB untuk menambah data, belum perlu berfungsi.
4. **Push ke repository,** dengan tangkapan layar di README kelompok.

CONTOH Contoh dari Laporkampus

KartuLaporan untuk entitas Laporan, LabelStatus untuk StatusLaporan, dan FAB Lapor di pojok kanan bawah. Datanya diambil dari sumber palsu Modul 2, jadi layar ini sudah memakai kode kelompok sendiri, bukan data tempelan.

Checkpoint minggu 3

No	Yang diselesaikan
1	LazyColumn berisi kartu entitas utama dan label status dari sealed class minggu lalu
2	Minimal satu filter atau urutan dengan fungsi koleksi, misalnya terbaru di atas
3	FAB tambah data, boleh belum berfungsi, dan tangkapan layar di README

Push ke repository kelompok. README memuat tangkapan layar Android atau desktop dan pembagian kerja. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16.

PENUTUP

Rangkuman

Compose bekerja secara **deklaratif**: kita mendeskripsikan tampilan untuk data tertentu, dan saat data berubah Compose memanggil ulang fungsinya. Fungsi yang menggambar tampilan diberi anotasi `@Composable`, hanya dipanggil dari `composable` lain, dan dinamai dengan PascalCase.

Layar disusun dari tiga wadah: **Column** dari atas ke bawah, **Row** dari kiri ke kanan, dan **Box** bertumpuk, dengan anak yang ditulis belakangan digambar paling atas. `weight` membagi sisa ruang di Row dan Column.

Modifier mengatur ukuran, jarak, latar, bingkai, dan interaksi, dan diterapkan berurutan. Padding sebelum background menjadi jarak di luar, padding sesudah background menjadi jarak di dalam.

Komponen yang baik menerima data lewat parameter, punya nilai bawaan untuk parameter opsional, dan melaporkan kejadian lewat lambda seperti `onClick` tanpa memutuskan sendiri. Fungsi koleksi seperti `filter`, `map`, `sortedBy`, dan `groupBy` mengolah daftar tanpa mengubah aslinya.

LazyColumn hanya menggambar item yang terlihat dan cocok untuk daftar panjang. Column dengan `forEach` cukup untuk daftar pendek yang jumlahnya pasti.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Apa yang terjadi pada tampilan Compose saat data yang dipakainya berubah?
2. Kalian ingin ikon, teks lokasi, dan waktu berjajar dalam satu baris, dengan waktu menempel di kanan. Wadah dan modifier apa yang dipakai?
3. Kenapa padding sebelum background berbeda hasilnya dengan padding sesudah background?
4. Di dalam Box, label ditulis lebih dulu, lalu foto dengan `fillMaxSize()`. Apa yang tampil?
5. Kenapa KartuLaporan menerima `onKlik: () → Unit`, bukan langsung membuka layar detail sendiri?
6. Kapan memilih LazyColumn, dan kapan Column dengan `forEach` sudah cukup?
7. Apa hasil `ListOf(5, 2, 8).sortedByDescending { it }.take(2)`?

JAWABAN Kunci jawaban

1. Composable yang memakai data itu dipanggil ulang, dan tampilannya ikut berubah. Proses ini disebut recomposition.
2. Row dengan `verticalAlignment = Alignment.CenterVertically`, dan teks lokasi diberi `Modifier.weight(1f)` supaya waktu terdorong ke kanan.
3. Modifier diterapkan berurutan. Padding pertama menjadi jarak di luar warna, padding setelah background menjadi jarak di dalam warna.
4. Hanya foto. Foto ditulis belakangan sehingga digambar paling atas, dan karena mengisi seluruh Box, label tertutup.
5. Supaya komponen bisa dipakai ulang di banyak tempat. Komponen hanya melaporkan kejadian, sedangkan pemanggilnya yang memutuskan apa yang dilakukan.
6. LazyColumn untuk daftar panjang atau yang jumlahnya tidak pasti. Column dengan `forEach` untuk daftar pendek yang jumlahnya pasti.
7. `[8, 5]`. Daftar diurutkan menurun menjadi 8, 5, 2, lalu diambil dua yang pertama.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
developer.android.com, Thinking in Compose	Cara berpikir deklaratif
developer.android.com, Compose layout basics	Column, Row, Box, dan arrangement
developer.android.com, Compose modifiers	Daftar modifier dan urutannya
developer.android.com, Lists and grids	LazyColumn, items, dan key
kotlinlang.org, Compose Multiplatform resources	Gambar dan berkas di composeResources
Kotlin Dasar P4 dan P5	Fungsi koleksi dan lambda

CONTOH Masalah Pertemuan 4

Kita membuat form laporan baru. Saat mahasiswa mengetik judul, kenapa teks di layar tidak ikut berubah? Dan kenapa isi form hilang saat layar diputar? Bawa layar daftar kelompok kalian. Mulai minggu depan kita fokus ke state dan arsitektur.